МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Казанский национальный исследовательский технический университет им. А.Н. Туполева-КАИ» (КНИТУ-КАИ)
Институт Компьютерных технологий и защиты информации

Кафедра Систем информационной безопасности

ОТЧЁТ

по лабораторной работе №4
по дисциплине
«Аппаратные средства телекоммуникационных систем»
на тему:

«ФУНКЦИОНАЛЬНАЯ ДЕКОМПОЗИЦИЯ С ГЛОБАЛЬНОЙ СУММОЙ И КРИТИЧЕСКИМИ СЕКЦИЯМИ»

Выполнил:
Студент гр. 4404
Устеников К. А.

Казань, 2026

Цель работы:

Освоить функциональную декомпозицию параллельной программы в сочетании с обновлением разделяемой (глобальной) величины, требующим явной синхронизации: на примере свёртки координат облака точек по формуле (Σx+Σy+Σz)/3N показать, как критическая секция предотвращает состояние гонки при конкурентном изменении общей суммы, и противопоставить этот подход редукции через разделение данных, применённому в предыдущей работе. Выполнена задача 6 пособия «Параллельное программирование на OpenMP».

Язык реализации — C#, директивы OpenMP в нём отсутствуют, поэтому конструкции моделируются штатными средствами. Соответствие для данной работы:

Конструкция OpenMP	Средство C# в работе
#pragma omp sections / section (функциональная декомпозиция)	три ветви кода (суммирование по осям x, y, z), распределяемые по потокам циклически (a % T == id)
#pragma omp critical (критическая секция)	блок lock (globalLock) { global += …; }
разделяемая глобальная сумма	переменная double global, захватываемая лямбдами потоков; без lock — гонка
неявный барьер конца sections	цикл Join()
reduction (альтернатива, не используемая по условию)	в работе сознательно заменена ручной критической секцией, чтобы продемонстрировать механизм синхронизации
OMP_NUM_THREADS / размер задачи	аргументы командной строки <threads> <N>

Задание:

Задача 6. Используя функциональную декомпозицию, переписать задачу 5 для вычисления значения по формуле (Σx+Σy+Σz)/3N, где N — количество точек. Использовать глобальную сумму и критические секции.

Общие требования. Число потоков и размер задачи передаются через командную строку. Библиотечные средства, решающие задачу целиком, запрещены. Каждый запуск подтверждается скриншотом с видимой командой и полным выводом.

Ход работы:

Программа читает из файла points.txt координаты N точек (файл формируется детерминированно и считывается обратно; числа — в инвариантной культуре, чтобы формат дроби не зависел от локали ОС) и раскладывает их в три плоских массива xs, ys, zs. В отличие от задачи 5, где вычислялся вектор геометрического центра (Σx/N, Σy/N, Σz/N), здесь требуется один скаляр — общее среднее по всем 3N координатам: (Σx+Σy+Σz)/3N. Для проверки корректности в главном потоке считается последовательный эталон expected, а после параллельной фазы результат сравнивается с ним (OK: True при совпадении).

Работа разбита на три функциональные ветви: суммирование всех x, всех y и всех z (аналог трёх #pragma omp section). Ветви распределяются по T потокам циклически (a % T == id), поэтому при T ≥ 3 активны ровно три потока, а остальные простаивают; при T < 3 одна ветвь исполняет несколько осей последовательно. Каждая завершённая ветвь обновляет единственную глобальную переменную global, прибавляя к ней свою частичную сумму. Поскольку к global concurrently обращаются разные потоки, это обращение заключено в критическую секцию lock (globalLock) { global += ls; } — прямой аналог #pragma omp critical. Без этой блокировки два потока могли бы одновременно прочитать global, каждый прибавить свою сумму и записать результат, потеряв одно из сложений (состояние гонки). Критическая секция вызывается один раз на выполненную ветвь (после локального накопления ls), поэтому накладные расходы на синхронизацию минимальны; сведение global/(3N) выполняется главным потоком после барьера Join().

Фрагмент вычислительного ядра (lab4/Program.cs):

```
double global = 0;
object globalLock = new object();
for (int t = 0; t < T; t++) {
    int id = t;
    th[id] = new Thread(() => {
        for (int a = 0; a < 3; a++) if (a % T == id) {
```

```
        double ls = 0;
        if (a == 0) for (int i = 0; i < N; i++) ls += xs[i];
        else if (a == 1) for (int i = 0; i < N; i++) ls += ys[i];
        else for (int i = 0; i < N; i++) ls += zs[i];
        lock (globalLock) { global += ls; }   // критическая секция
    }
  });
}
```

Команды запуска (N = 1000000):

```
cd lab4
dotnet run -- 2 1000000
dotnet run -- 4 1000000
dotnet run -- 8 1000000
```

На рисунке 1 — запуск с 2 потоками: виден скалярный результат, совпадение с эталоном (OK: True) и время параллельной фазы.

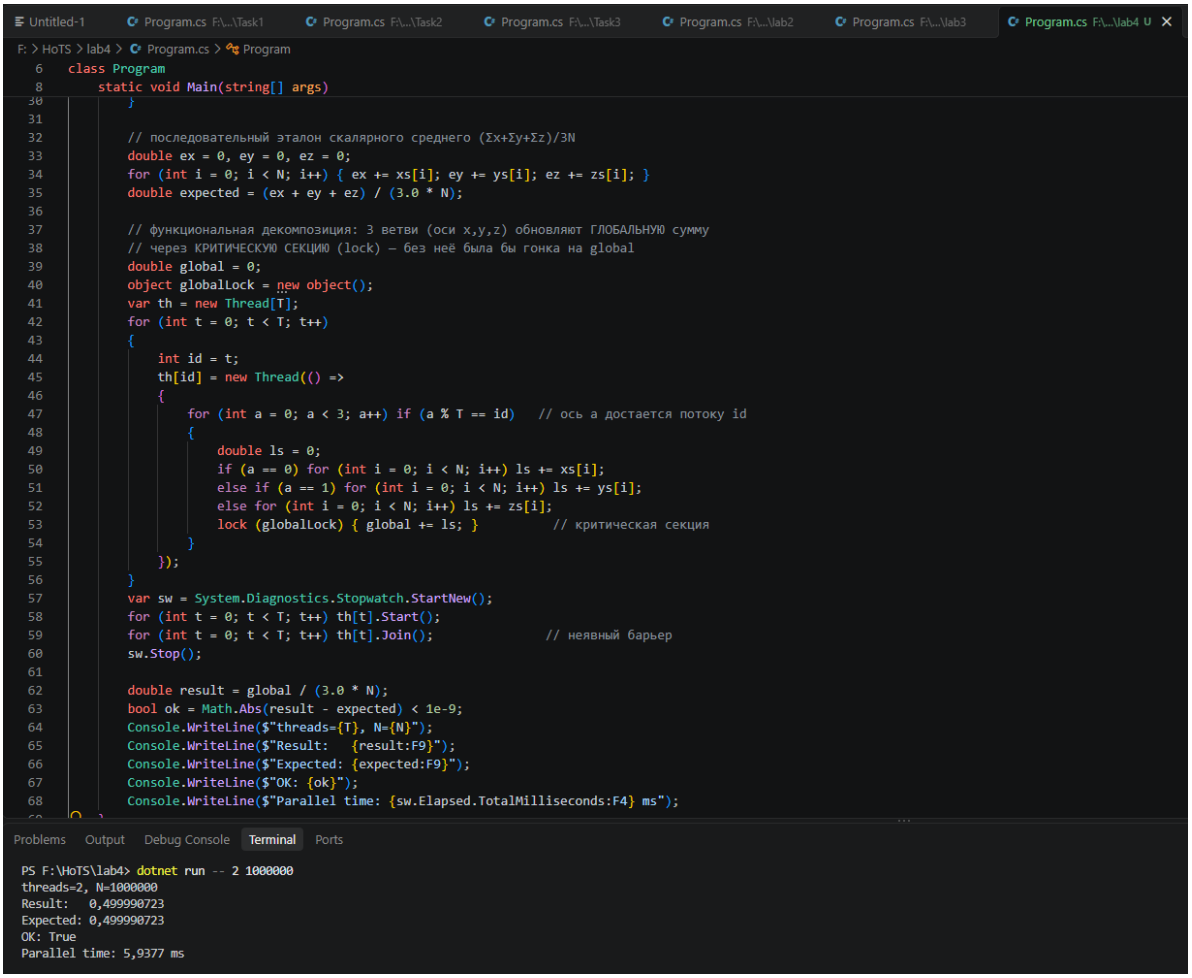


Рисунок 1. Функциональная декомпозиция, 2 потока

На рисунке 2 — 4 потока: три ветви исполняются параллельно, четвёртый поток простаивает; результат и признак совпадения неизменны.

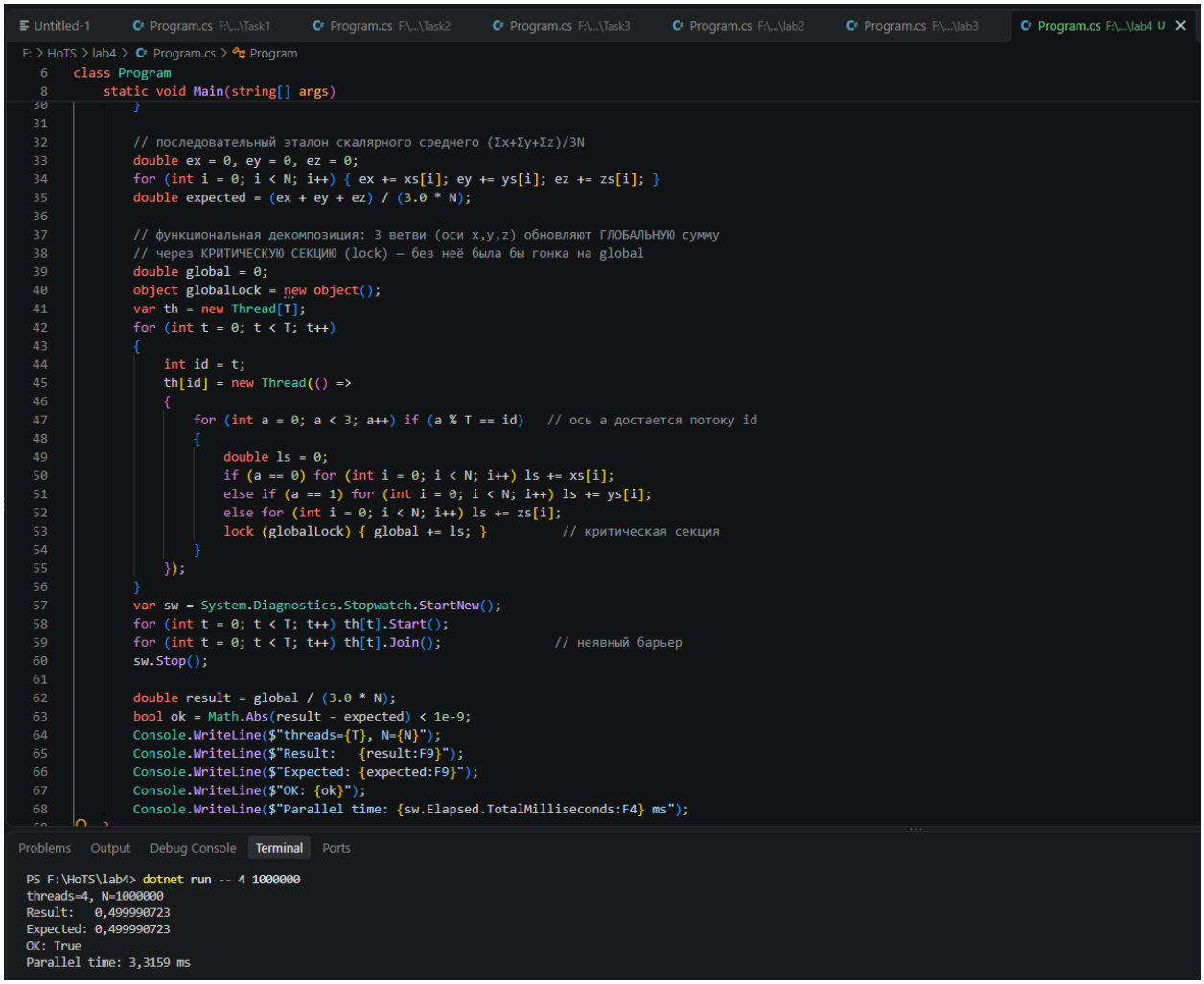


Рисунок 2. Функциональная декомпозиция, 4 потока

На рисунке 3 — 8 потоков: активны те же три ветви, остальные простаивают; lock сериализует три обновления глобальной суммы, результат не зависит от числа потоков.

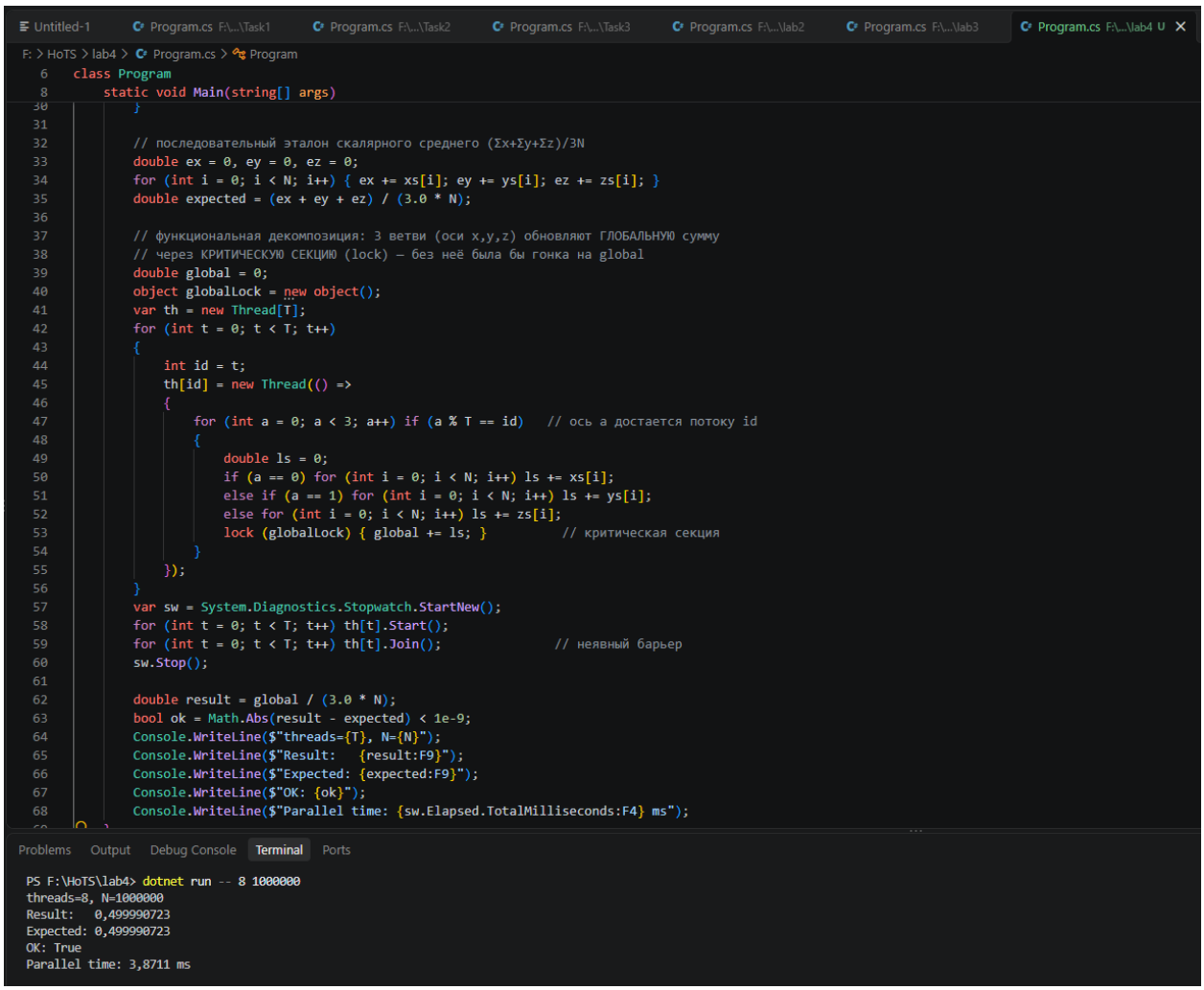


Рисунок 3. Функциональная декомпозиция, 8 потоков

Значение результата идентично во всех запусках, что подтверждает корректность критической секции: независимо от порядка вхождения потоков в lock, глобальная сумма собирается одинаково.

Число потоков	Результат	Ожидаемый	OK	Время, мс
2	0.499990723	0.499990723	True	5.9377
4	0.499990723	0.499990723	True	3.3159
8	0.499990723	0.499990723	True	3.8711

Время параллельной фазы убывает при переходе от 2 к 4 потокам (выход из дисбаланса размещения трёх ветвей) и не уменьшается далее (4 → 8 потоков), оставаясь в пределах шума: функциональных ветвей ровно три (оси координат), поэтому параллелизм ограничен min(T, 3), а при T > 3 лишние

потоки простаивают и добавляются лишь накладные расходы на их создание и на входы в критическую секцию. Ключевой факт — неизменность значения Result при любом T, что подтверждает корректность lock: критическая секция сериализует обновления глобальной суммы, делая итог детерминированным.

Вопросы:

1. Чем задача 6 отличается от задачи 5 по способу распараллеливания?

Потому что разделяемая запись отсутствует по построению. Каждый поток В задаче 5 (версия с редукцией) частичные суммы складывались в непересекающиеся слоты потоков, а итог подводил главный поток после барьера — разделяемой записи не было, поэтому синхронизация не требовалась. В задаче 6 условие предписывает глобальную сумму, которую конкурентно обновляют все ветви, поэтому необходима критическая секция lock, серилизующая каждое прибавление. Кроме того, задача 5 выдаёт вектор центра (Σx/N, Σy/N, Σz/N), а задача 6 — скаляр (Σx+Σy+Σz)/3N.

2. Что произойдёт, если убрать lock?

Возникнет состояние гонки на переменной global: операция global += ls не атомарна (чтение → сложение → запись), и при наложении по времени двух ветвей одно из сложений будет потеряно, итог станет недетерминированным и, как правило, меньше истинного. Наличие lock гарантирует, что в критический блок в каждый момент входит только один поток, поэтому все три прибавления учитываются и OK: True сохраняется при любом числе потоков.

3. Почему критическая секция вызывается один раз на ветвь, а не на каждую итерацию?

Потому что каждая ветвь сначала накапливает частичную сумму в локальную переменную ls (приватную для потока), а в общий global входит лишь один раз по завершении ветви. Если бы lock стоял внутри цикла по точкам, блокировка дёргалась бы 3N раз, конкуренция за неё сделала бы параллельную версию медленнее последовательной. Именно от ручных блокировок на каждой итерации в OpenMP избавляет условие reduction; в

настоящей работе reduction сознательно не применяется, чтобы продемонстрировать работу критической секции, как того требует условие задачи 6.

Вывод: Реализована функциональная декомпозиция вычисления скалярной свёртки (Σx+Σy+Σz)/3N по трём ветвям (оси координат) с обновлением единой глобальной суммы; корректность подтверждена совпадением с последовательным эталоном (OK: True во всех запусках).

Продемонстрирована необходимость критической секции lock (аналог #pragma omp critical) при конкурентном изменении разделяемой переменной: без неё возникает потеря сложений (состояние гонки), с ней результат детерминирован при любом числе потоков.

Сопоставлено с задачей 5: там редукция достигалась разделением данных без блокировок, здесь же условие требует глобальной суммы, поэтому введена явная синхронизация; критическая секция вызывается один раз на ветвь после локального накопления, что минимизирует накладные расходы и показывает, зачем в OpenMP существует механизм reduction.